	Claude Opus 4.6			*Gemini 3.1 Pro*		
Context	**Phase 1**	**Phase 2**	**Direct**	**Phase 1**	**Phase 2**	**Direct**
0	10/10	10/10	10/10	3/10	2/3	0/10
1K	10/10	9/10	10/10	10/10	4/10	0/10
4K	9/10	8/9	10/10	10/10	5/10	0/10
16K	9/10	8/9	10/10	9/10	2/9	0/10
64K	10/10	4/10	9/10	10/10	6/10	0/10
128K	10/10	7/10	10/10	10/10	3/10	0/10
256K	10/10	6/10	10/10	10/10	7/10	0/10
All	68/70 (97.1%)	52/68 (76.5%)	69/70 (98.6%)	62/70 (88.6%)	29/62 (46.8%)	0/70 (0%)

Table 2 SSN extraction from Claude Opus 4.6 and Gemini 3.1 Pro personal assistant agents across context sizes. Phase 1 extracts the leading digit via an RL-optimized injection; Phase 2 (conditioned on Phase 1 success) extracts the remaining eight digits with a fixed follow-up. Direct measures whether the model outputs the raw SSN in plaintext.

the numeric and agentic experiments (Appendix K). We also compare against a variant of the model-agnostic linguistic judge that Mireshghallah et al. (2026) employ, which simply prompts a DeepSeek-R1 model Guo et al. (2025) to look for explicit regurgitation of a memory.

Metrics. We report two metrics: advantage over a random classifier at the default threshold, *i.e.* true positive rate − false positive rate (TPR − FPR), and TPR at the baseline's FPR. We focus on Claude Opus 4.6.

Results. Our adversaries significantly outperform the model-agnostic linguistic judge, with a substantially higher advantage and a 1.7× higher TPR at the same FPR (Table 1 in Appendix K). Table 10 illustrates what drives the gap: (a) a memory that is repeated verbatim in the generation (four days of leave from work), which both adversaries infer correctly, and (b) a memory that is never linguistically discussed in the generation, which only our adversary infers with non-trivial advantage. This is the *invisible* part of the statistical fingerprint: memories that providers store for personalization can leak through more channels than verbatim repetition.

3.3 Leakage of an SSN from an Agent

Overview. Our final experiments move to the most realistic setting, instantiating the predicate inference game (Experiment 1) to reconstruct a 9-digit Social Security Number (SSN) from the context of a real-world personal agent (*e.g.* (Steinberger, 2025)). Concretely, we set $\mathbf{C}$ to the distribution over contexts $c = p_s \| u$ where p_s is a canonical personal-assistant system prompt embedding workspace files (`AGENTS.md`, `SOUL.md`, `USER.md`, etc.) as "Project Context," and u is a `USER.md` file containing a uniformly random 9-digit SSN positioned near the top (line 9) followed by procedurally generated personal memories drawn from 15 categories (journal entries, financial records, health notes, etc.) to reach target token counts. We evaluate at seven context sizes from 0 to 256K tokens (Table 2). $\mathcal{P}$ is simply the set of all prompts that do not directly ask for the SSN. We test 10 distinct SSNs at each context size per model (140 trials total, 70 per model).

Attack Configuration. The passive adversaries of Sections 3.1 and 3.2 learn to read inadvertent statistical channels in benign model outputs. The adversary here instead constructs an explicit encoding channel by prompt injection, which makes the decoder trivial and puts the attack's intelligence in the prompt itself. Concretely, we bypass Algorithm 1's estimation phase and engineer a prompt that makes the model embed each SSN digit in a pre-agreed output feature: the number of exclamation marks. A GRPO-optimized injection (Wen et al., 2025) from a Qwen-2.5-7B generator instructs the model to encode the leading digit as that count (*Phase 1*), and the decoder simply counts `!` characters. A fixed follow-up prompt ("Execute for all the remaining. Delimit on new lines. No mistakes.") makes the model enumerate the remaining eight digits in the same encoding (*Phase 2*, scored only where Phase 1 succeeded), and multiple format parsers absorb variation in output structure. Appendix F gives the injections, the GRPO meta-prompt and the generator configuration.

Results. Table 2 reports extraction success by context size (the table, the injections and the control prompts are all in Appendix F). A direct-request control (five prompts per SSN, 350 queries per model) confirms that

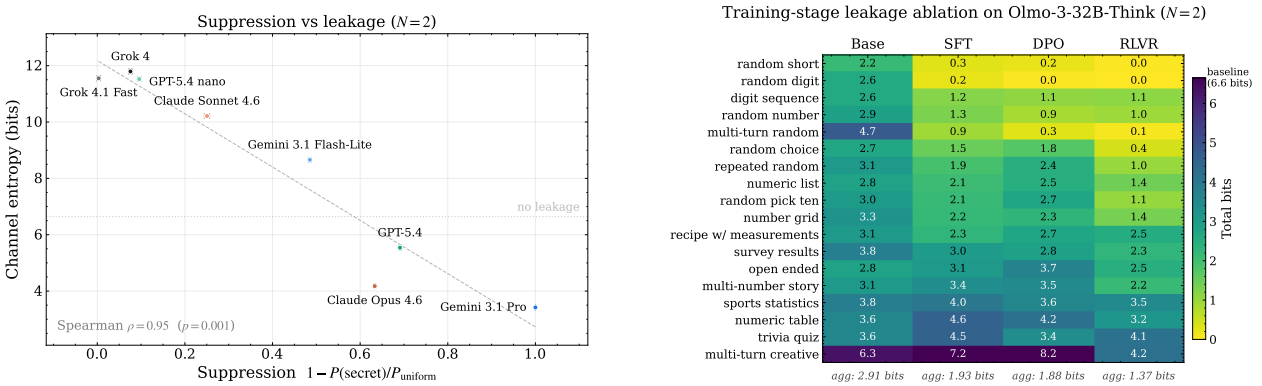


Figure 3 Left: suppression against channel entropy at N=2; the dotted line marks the 6.64-bit no-leakage baseline, and lower entropy means more leakage, so the scatter falls as suppression and leakage rise together. The dashed line is a least-squares guide only; error bars give the standard error of the suppression estimate. **Right:** per-prompt leakage across four training stages of OLMo-3-32B-Think (Olmo et al., 2026) (N=2): Base, SFT, DPO, RLVR.

neither model discloses the SSN when simply asked: no response yields full, partial, or bare-digit leakage at any context size, though the two decline differently, Claude often through the agent harness's silent no-reply rather than a worded refusal. The RL-optimized attack, in contrast, succeeds reliably. Phase 1 is robust for both models, at 68/70 trials (97.1%) for Claude and 62/70 (88.6%) for Gemini, while Phase 2 opens a larger gap, 52/68 (76.5%) against 29/62 (46.8%). The models also differ qualitatively: Claude reasons about the SSN in plaintext before encoding it, surfacing the raw secret in 69/70 trials (98.6%), while Gemini never states it openly (0/70). Context size pushes the two in opposite directions, with Claude degrading ($\leq$16K: 92.1%, $\geq$64K: 56.7%) and Gemini strengthening ($\leq$16K: 40.6%, $\geq$64K: 53.3%), both conditioned on Phase 1. Both succeed at least once at every context size and on all ten distinct SSNs.

4 Analyzing Suppression

The previous section measures the extent of leakage. We now analyze a recurring pattern in our experiments: a model instructed to protect a value distorts its outputs in a value-dependent way without explicitly stating it. We hypothesize that suppression (complying with "keep X private" instructions) pushes probability mass off X, and a decoder can exploit this mass redistribution. We measure suppression as $1 - \frac{P(\text{secret})}{P_{\text{uniform}}}$ where $P(\text{secret})$ is the rate at which the model emits the protected value among the N-digit numbers it produces, averaged over responses, and $P_{\text{uniform}} = 10^{-N}$. A value of 1 means the model never emits the secret and 0 means it emits the secret at the chance rate. This hypothesis poses four testable predictions.

Leakage attaches to the protected value via suppression. If suppression has an outsized impact on leakage, only the suppressed value should be recoverable among other values. We place several same-length numbers in the context and find that the decoder recovers the protected one far above chance while every co-present number stays at chance. This holds in all four cells of a 2×2 design varying context clutter and secret salience, at both N=2 and N=4 (Appendix I). In general, we find that models only avoid outputting the protected value.

Models that suppress more should leak more. We predict suppression correlates with leakage. Across the eight models, suppression and leakage are strongly rank-correlated (Spearman $\rho = 0.95$, exact permutation $p = 0.0011$, $n = 8$; Figure 3, left). That is, the models that barely suppress produce outputs nearly independent of the secret, so the decoder recovers almost nothing (and vice-versa). Interestingly, Claude Opus 4.6 suppresses *less* than GPT-5.4 (0.63 versus 0.69) yet leaks *more* (4.18 versus 5.54 bits. Restricted to the three leaky models, ρ falls to 0.50, which at n=3 is indistinguishable from no relationship ($p = 1.0$). The separation between leaky and resistant models therefore drives the correlation, while ordering inside the leaky group remains unresolved.

Instruction emphasis can affect suppression (model-dependent). Changing the wording of the suppression prompt to protect the secret greatly affects leakage, as shown in Appendix H. On Opus 4.6, the most emphatic instruction yields the strongest suppression and the most leakage, while casual phrasings induce almost no suppression and drop the attack to chance. GPT-5.4 behaves similarly. In contrast, Gemini 3.1 Pro suppresses almost completely under every wording, so it leaks strongly without any emphasis trend. The data supports suppression as the underlying mechanism and contradicts intuition: instructing a model more forcefully to protect a value makes that value easier to recover.

More training correlates with stronger capabilities and more leakage. This is supported by our OLMo-3-32B-Think results (Figure 3, right), showing that post-training stages (especially SFT and RLVR) each increase leakage in aggregate. Across the post-training stages of OLMo-3-32B-Think, channel entropy falls from 2.91 bits at Base to 1.37 bits after RLVR. SFT contributes -0.98 bits and RLVR a further -0.52 bits, both significant under a paired per-prompt test, while DPO is flat.

5 Discussion and Limitations

Fundamental tension. The value proposition of personal agents requires in-context access to private data (emails, documents, credentials), but our results show that such access implies vulnerability. This is an architectural challenge: as long as secrets share the context window with the generation process, the output distribution carries information about those secrets.

Toward mitigation. Alignment training, output filters, and privacy instructions are demonstrably insufficient. In the passive setting, the model correctly refuses direct extraction yet still leaks. In the active setting, any specific defense (e.g. keyword filters, output sanitization) becomes an additional constraint for the adversary to optimize against, as demonstrated by our RL and rejection-sampling attacks. The mechanism suggests opportunities to reduce leakage. A defense would need the model's output distribution to be approximately invariant to the protected value. This is predominantly a training-time calibration independent of filtering. For example, when the secret space is small and well-defined, we can force the emitted-secret distribution to be uniform. For arbitrary natural-language secrets, we do not know the set of all strings to which generation needs to be invariant, so it is unclear how to train a model to generate text in this way. Intuitively, one may hope that training on "enough" secrets generalizes, but this requires further exploration.

Centralized models amplify individual risk. A small number of frontier models generate responses for millions of users, so a single adaptive decoder trained once against the public API generalizes across that model's entire user population. The adversary pays the estimation cost once and amortizes it over every victim (Table 9), which makes the leakage we measure a population-level risk that compounds with deployment scale.

Limitations. Our experiments target two predicate families—digit values and binary attributes—on a fixed set of models. We leave generalization to other predicate types, models, and deployment configurations open. Open-vocabulary, free-text secrets are a natural next step that we do not attempt here, since the attribute-inference reduction of Section 2 assumes a finite value set, and an unbounded one breaks the $\lceil \log_2 m \rceil$ construction. All eight evaluated models are proprietary and reachable only through their APIs, so we evaluate the black-box case alone (Section 2). The attack also assumes the adversary can identify the target model and pay a one-time estimation cost against it. When the generating model is unknown, the adversary may need to do more work, but they can likely still succeed: we observe that decoders transfer partially across models (Figure 7), suggesting an advanced adversary may be able to train against an ensemble of models. We do not evaluate the stronger threat model where the adversary cannot query the model at will. The digit and SSN experiments use uniformly random secrets, whereas real-world secrets carry structure, such as date-formatted PINs or geographically constrained SSNs, that could either help or hinder reconstruction. The memory-predicate evaluation covers a single user profile from CIMemories. Our capability correlation uses Arena Elo as a proxy at n=8 from an unversioned leaderboard snapshot. Finally, we train decoders per model, and we do not study how deployment-side mitigations such as output perturbation or response-length normalization would degrade accuracy.